UNIT-5

IntroductiontopatternMatching

It'stheprocessofidentifyingspecificsequencesofcharactersor elements within a larger structurelike text, data, or images. Think of it like finding a specific word in a sentence or a sequenceofsymbolsorvalues,withinalargersequenceor text.

Basic Concepts Pattern Matching

- **Pattern:** A pattern is a sequence of characters, symbols, or other data that forms a search criterion. In text processing, a pattern could be a string of characters.
- **Text:** The text (or string) is the sequence where the pattern is searched for.
- **Match:** A match occurs if the pattern is found within the text. The goal of pattern matching is to find all instances where this occurs or to determine whether the pattern exists in the text.



Basic Concepts Pattern Matching

- **Pattern:** A pattern is a sequence of characters, symbols, or other data that forms a search criterion. In text processing, a pattern could be a string of characters.
- **Text:** The text (or string) is the sequence where the pattern is searched for.
- **Match:** A match occurs if the pattern is found within the text. The goal of pattern matching is to find all instances where this occurs or to determine whether the pattern exists in the text.



Brute Force Pattern Matching Algorithm

Checks for the pattern at every possible position in the text. For each position, it compares the pattern with the corresponding substring in the text. It is Effective for small texts or patterns, but inefficient for large texts.

Knuth-Morris-Pratt(KMP)

Optimizes the naive approach by avoiding redundant comparisons. It pre-processes the pattern to determine the longest prefix which is also a suffix, allowing the search to skip some comparisons. It is Suitable for applications where the same pattern is searched repeatedly in multiple texts.

Boyer-Moore

Works by comparing the pattern to the text from right to left. It uses two heuristics, the bad character rule and the good suffix rule, to skip sections of the text, offering potentially sub-linear time complexity. It is Highly efficient for large texts and is considered one of the fastest single-pattern matching algorithms.

ⓘ X

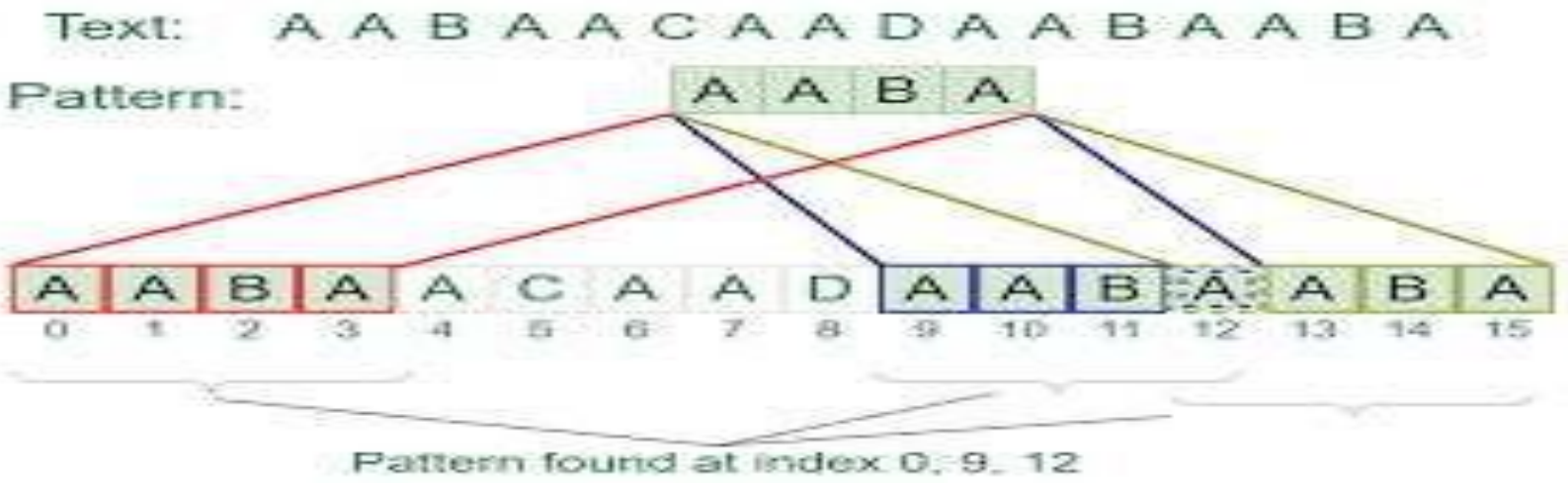
Brute force Pattern Matching

A brute force algorithm is a straight forward approach to solving a problem. It also refers to a programming style that does not include any shortcuts to improve performance.

- It is based on **trial and error** where the programmer tries to merely utilize the computer's fast processing power to solve a problem, rather than applying some advanced algorithms and techniques developed with human intelligence.
- It might increase both space and time complexity.
- A simple example of applying brute force would be linearly searching for an element in an array. When each and every element of an array is compared with the data to be searched, it might be termed as a brute force approach, as it is the most direct and simple way one could think of searching the given data in the array

Brute Force Pattern Matching Algorithm

1. Start at the beginning of the text and slide the pattern window over it.
2. At each position of the text, compare the characters in the pattern with the characters in the text.
3. If a mismatch is found, move the pattern window one position to the right in the text.
4. Repeat steps 2 and 3 until the pattern window reaches the end of the text.
5. If a match is found (all characters in the pattern match the corresponding characters in the text), record the starting position of the match.
6. Move the pattern window one position to the right in the text and repeat steps 2-5.
7. Continue this process until the pattern window reaches the end of the text.



```
function bruteForcePatternMatch(T, P):
    n = length(T)
    m = length(P)

    for i from 0 to n - m:
        j = 0
        while j < m and T[i + j] == P[j]:
            j = j + 1
        if j == m:
            return i  // Pattern found at position i
    return -1  // Pattern not found
```

Boyer–Moore Pattern Matching

The **Boyer–Moore Pattern Matching algorithm** is one of the most efficient string-searching algorithm that is the standard benchmark for practical pattern matching. It was developed by **Robert Stephen Boyer** and **J Strother Moore** in the year **1977**.

The Boyer-Moore algorithm works by **pre-processing** the pattern and then scanning the text from right to left, starting with the rightmost characters. It is based on the principle that if a mismatch is found, there is no need to match the remaining characters. This backwards approach significantly reduces the algorithm's time complexity compared to naive string search methods.

Here's a step-by-step explanation of how the Boyer-Moore algorithm works:

1. Preprocessing

Boyer-Moore algorithm uses two heuristics for preprocessing the pattern:

a. Bad Character Heuristic

b. Good Suffix Heuristic

Bad Character Heuristic (Building Bad Character Table)

Determines how far the pattern can be shifted when a character mismatch occurs.

a. Create an array (often called the "**Bad Character Table**") to store the rightmost occurrence of each character in the pattern. If a character is not in the pattern, its value is set to **pattern length**.

b. For each character in the pattern, update the table with its last occurrence in the pattern with **pattern length - index - 1**.

$$\text{BCT(ch)} = \begin{cases} \text{m} & if \text{ ch is not in p} \\ \text{m-j-1} & \text{otherwise} \end{cases}$$

Example:

j →	0	1	2	3	4
P	M	I	S	S	I

Length(m)=5

Bad Character Table

M	I	S	*
4	0	1	5

M → (m-j-1) = 5-0-1 = 4

I → (m-j-1) = 5-1-1 = 3

S → (m-j-1) = 5-2-1 = 2

S → (m-j-1) = 5-3-1 = 1

I → (m-j-1) = 5-4-1 = 0

2. Searching

Once the preprocessing is done, the actual search begins:

a. Align the pattern with the beginning of the text.

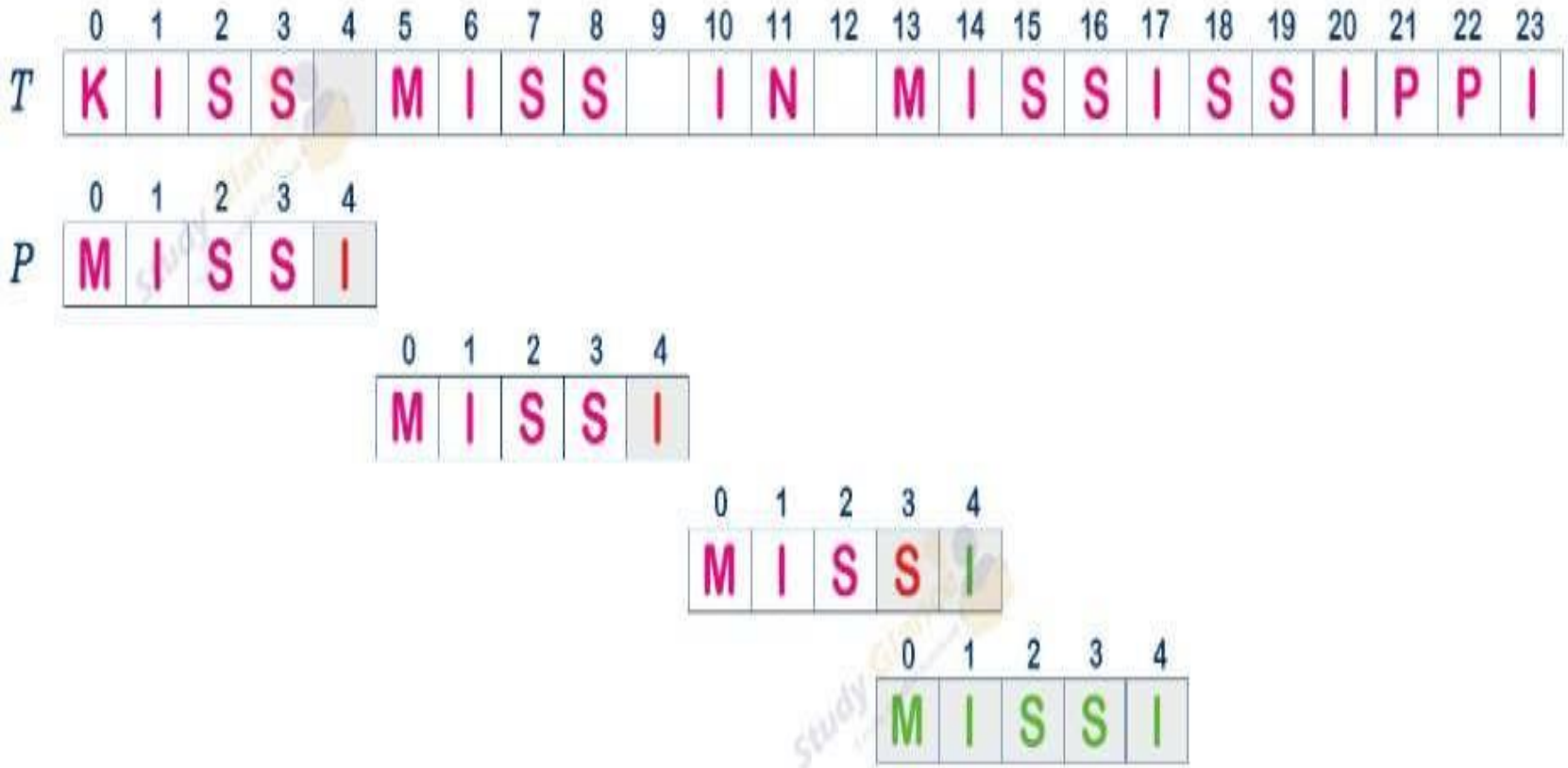
b. Compare the pattern with the text from right to left.

c. If all characters of the pattern match, a valid occurrence is found.

3. Shifting the Pattern:

When a mismatch occurs

- **If the character is not in the pattern:** Shift the pattern such that the mismatched character in the text aligns with the rightmost occurrence of it in the pattern.
- **If the character is not in the pattern:** shift the entire pattern past the character.


T
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
K I S S M I S S I N M I S S I S S I P P I
P
0 1 2 3 4
M I S S I
0 1 2 3 4
M I S S I
0 1 2 3 4
M I S S I
0 1 2 3 4
M I S S I

3. Repeat Comparison

After shifting the pattern according to the above rules, repeat the comparison process:

a. Continue comparing the pattern with the text from right to left.

b. Apply the shifting rules whenever a mismatch is encountered.

c. Continue this process until the end of the text is reached or all occurrences of the pattern are found.

4. Termination

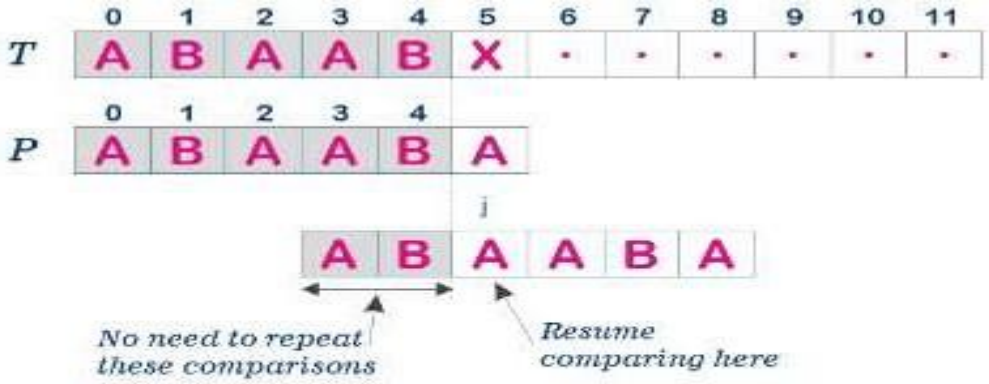
The algorithm terminates when either

- The pattern has been shifted past the **end of the text**, indicating no more matches are possible.
- All occurrences of the **pattern has have been found**.

KMP Pattern Matching

The **Knuth-Morris-Pratt (KMP)** pattern matching algorithm is an efficient string searching method developed by **Donald Knuth**, **James H. Morris**, and **Vaughan Pratt** in the year 1970. It is used to find the occurrences of a "pattern" within a "text" without checking every single character in the text, which is a significant improvement over the brute-force approach.

The KMP algorithm compares the pattern to the text in left-to-right, but shifts the pattern, P more intelligently than the brute-force algorithm. When a mismatch occurs, what is the most we can shift the pattern so as to avoid redundant comparisons. The answer is that the largest prefix of P[0..j] that is a suffix of P[1.j].

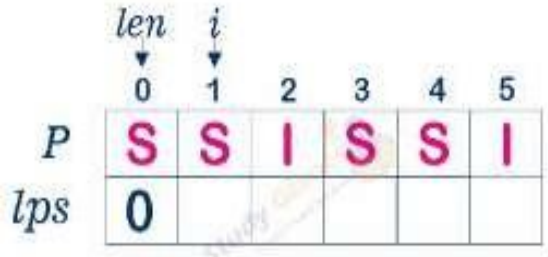


Here's a step-by-step explanation of how the KMP algorithm works:

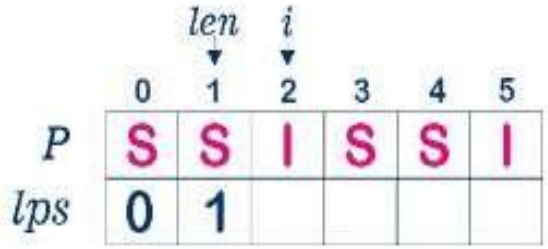
1. Preprocessing(Building the LPS Array)

The core idea to preprocess the pattern to construct an **LPS(Longest Prefix Suffix)** array. This array stores the length of the longest proper prefix which is also a suffix for each sub-pattern of the pattern. This preprocessing helps in determining the next positions in the pattern to be compared, thus avoiding redundant comparisons.

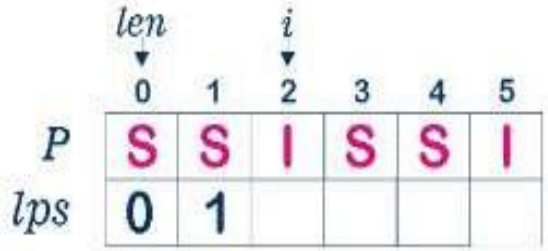
1. Start by initializing the first element of **lps[]** to 0, as a single character can't have any proper prefix or suffix.
2. Maintain two pointers, **len** and **i**, where **len** is the length of the last longest prefix suffix. Initially, **len := 0** and **i := 1**.
3. Repeat steps 4 to 6, **while (i < m)**
4. If **pattern[len]** equals **pattern[i]**, set **lps[i] = len + 1**, increment both **i** and **len**.
5. If they don't match and len is not 0, update len to **lps[len - 1]**.
6. If they don't match and len is 0, set **lps[i] = 0** and increment i.



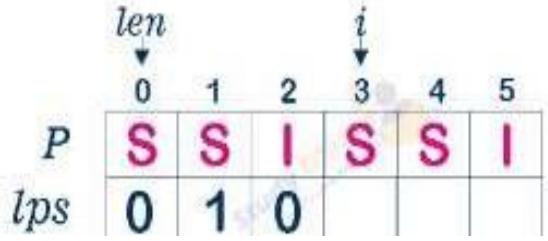
lps[0]=0



if(p[len] = p[i]) then
lps[i] = len+1
increment len and i



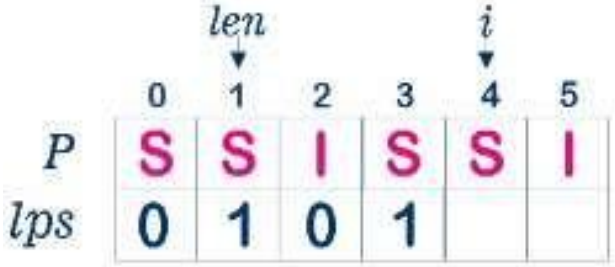
if(p[len] ≠ p[i] and len > 0) then
len = lps[len-1]



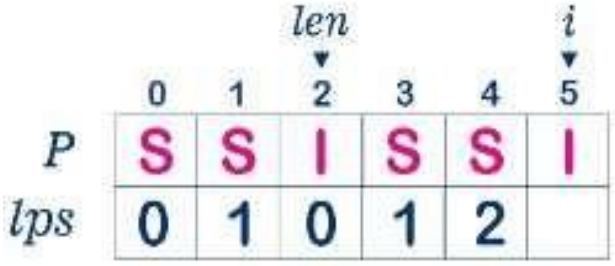
if(p[len] ≠ p[i] and len = 0) then
lps[i] = 0
increment i



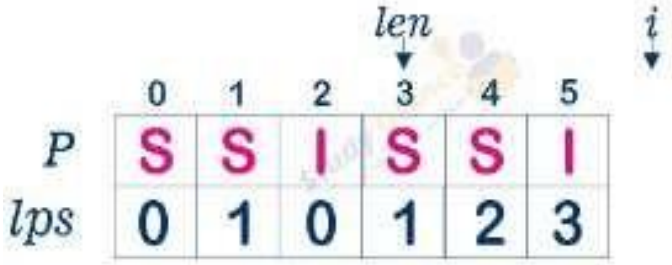
if(p[len] ≠ p[i] and len = 0) then
lps[i] = 0
increment i



if(p[len] = p[i]) then
lps[i] = len+1
increment len and i



if(p[len] = p[i]) then
lps[i] = len+1
increment len and i



if(p[len] = p[i]) then
lps[i] = len+1
increment len and i

2. Searching

2. Searching

Once the preprocessing is done, the actual search begins:

a. Align the pattern with the beginning of the text.

b. Compare the pattern with the text from left to right.

c. If all characters of the pattern match, a valid occurrence is found.

3. Shifting the Pattern:

- Compare pattern[j] with text[i].
- If they match, increment both i and j.
- If j equals the pattern length, a match is found. Optionally report the match, then set j to lps[j - 1].
- If they don't match and j is not 0, set j to lps[j - 1]. Do not increment i here.
- If they don't match and j is 0, increment i

4. Repeat Comparison

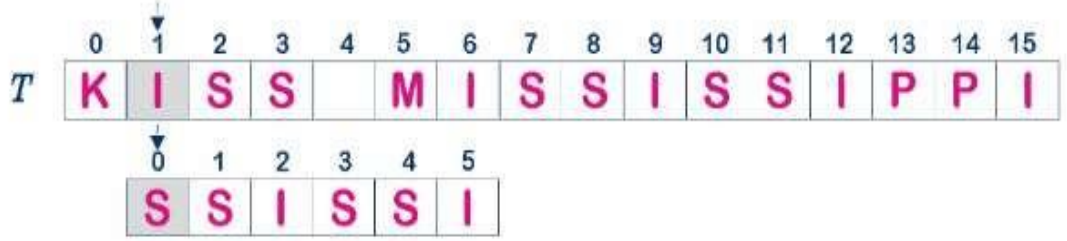
a. Continue comparing the pattern with the text from left to right.

b. Apply the shifting rules whenever a mismatch is encountered.

c. Continue this process until the end of the text is reached or all occurrences of the pattern are found.



if(P(j) ≠ T(i) and j = 0) then increment i



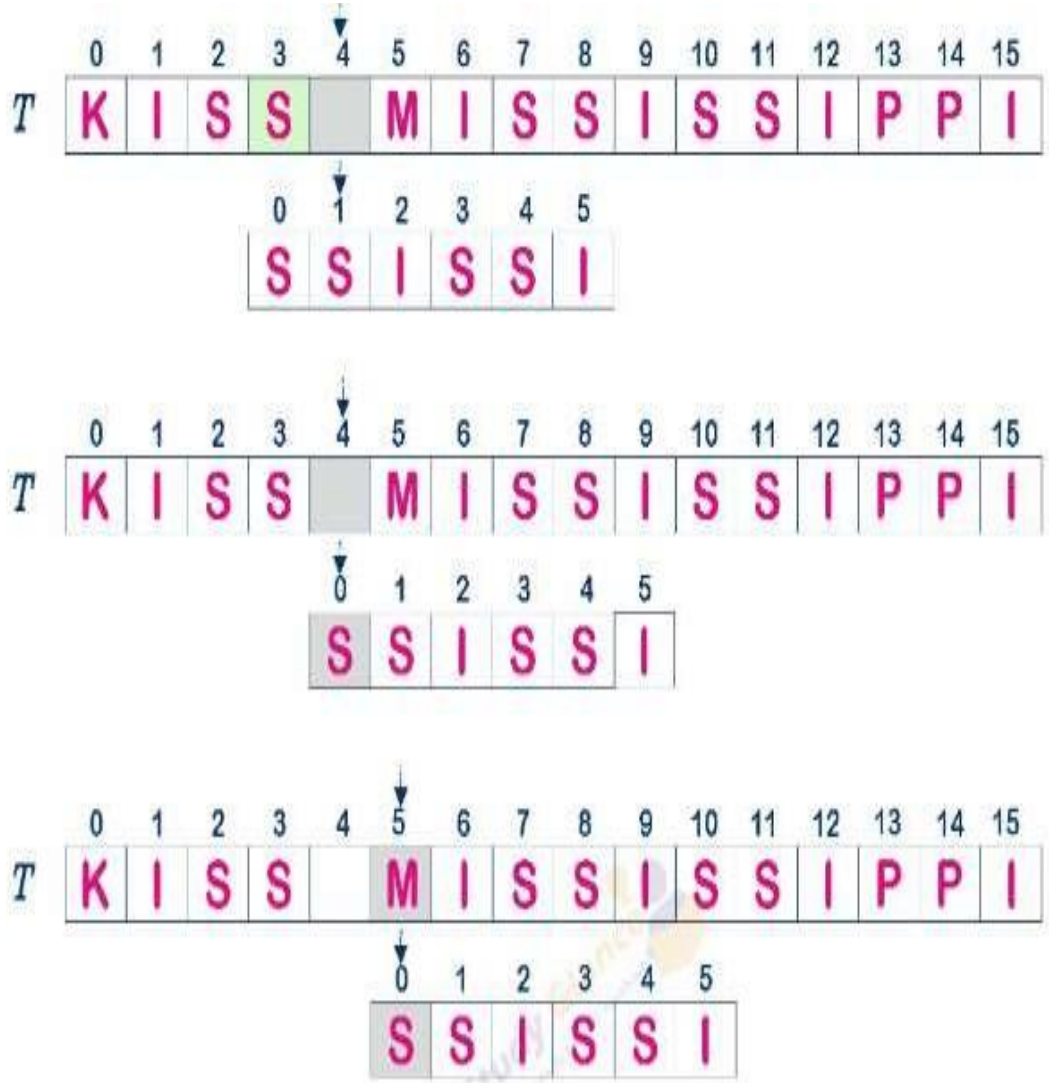
if(P(j) ≠ T(i) and j = 0) then increment i

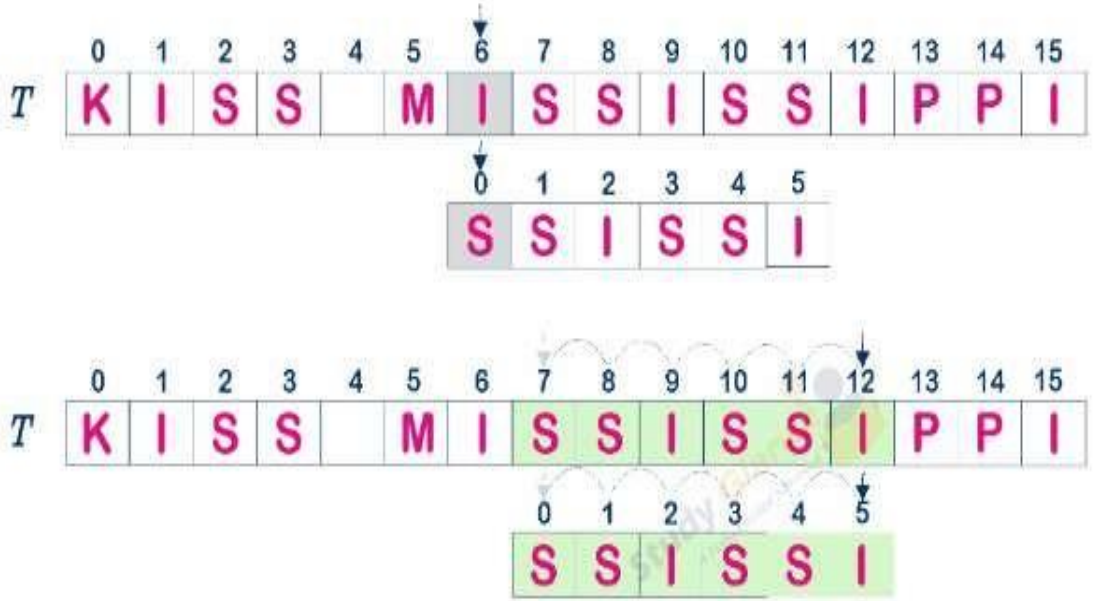


if(P(j) = T(i) and j ≠ m-1) then increment i and j

if(P(j) ≠ T(i) and j > 0) then j = lps[j-1]

Computer data storage


T
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
K I S S M I S S I S S I P P I
0 1 2 3 4 5
S S I S S I
if(P(j) ≠ T(i) and j > 0) then
j = lps[j-1]
T
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
K I S S M I S S I S S I P P I
0 1 2 3 4 5
S S I S S I
if(P(j) ≠ T(i) and j = 0) then
increment i
T
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
K I S S M I S S I S S I P P I
0 1 2 3 4 5
S S I S S I
if(P(j) ≠ T(i) and j = 0) then
increment i



5. Termination

The algorithm terminates when either

- The pattern has been shifted past the **end of the text**, indicating no more matches are possible.
- All occurrences of the **pattern has have been found**.

Pseudo code for KMP Pattern Matching algorithm

Microsoft PowerPoint

ConstructaLPSArray

```
Len=0,i=1,lps[0]=0
While(i<m)
{
if(p[len]==p[i])
{
lps[i]=len+1;i++;len++;
}
elseif(p[len]!=p[i]&&l>0)
{
Len=lps[len-1];
}
else(p[len]!=p[i]&&l=0)
{
 lps[i]=0,i++
}
}
```

Kmppatternmatching

i=0,j=0;

if(p[j]==t[i]&&j!=m-1)

i++,j++

elseif(p[j]!=t[i]&&j>0)

J=lps(j-1);

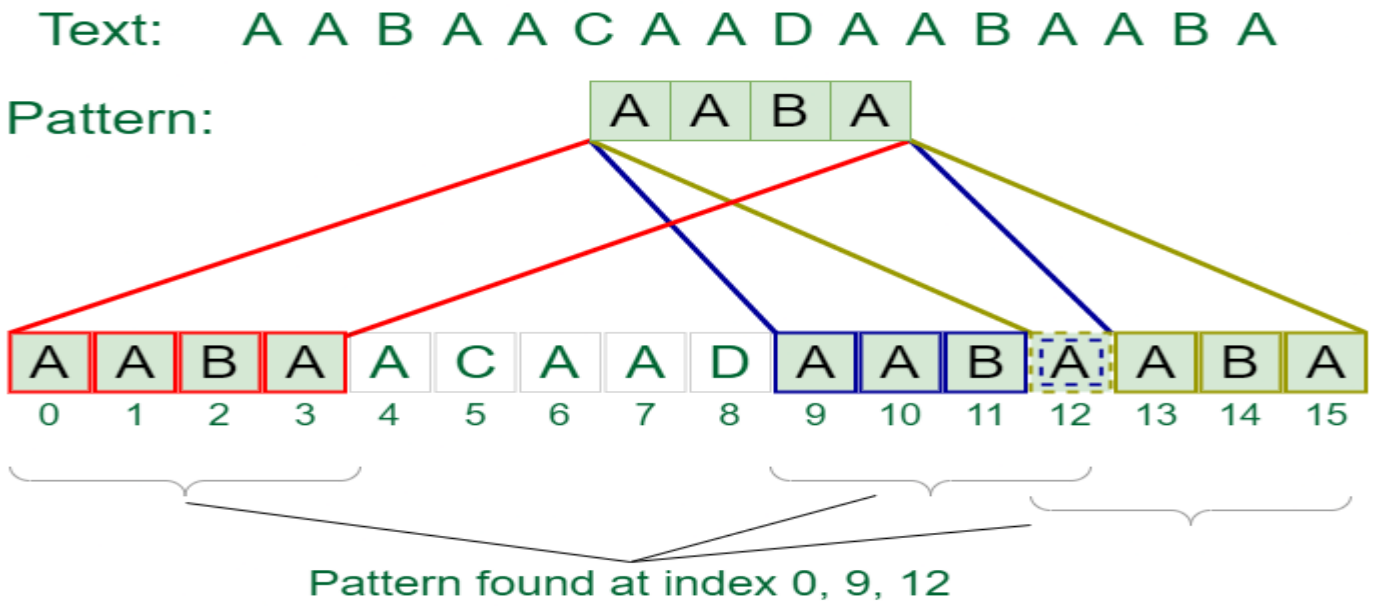
elseif(p[j]!=t[i]&&j=0) i++

else(p[j]==t[i]&&j=m-1)

Pattern found

Naive algorithm for Pattern Searching

- Given **text** string with length **n** and a **pattern** with length **m,** the task is to prints all occurrences of **pattern** in **text**.
Note: You may assume that n > m.
Examples:
Input: *text = "THIS IS A TEST TEXT", pattern = "TEST"*
Output: *Pattern found at index 10*
Input: *text = "AABAACAADAABAABA", pattern = "AABA"*
Output: *Pattern found at index 0, Pattern found at index 9, Pattern found at index 12*



Pattern searching

Naive Pattern Searching algorithm:

Slide the pattern over text one by one and check for a match. If a match is found, then slide by 1 again to check for subsequent matches.

Rabin-Karp Algorithm

The Rabin-Karp Algorithm is a string-searching algorithm named after its authors, Michael O. Rabin and Richard M. Karp.

The Rabin-Karp Algorithm is an algorithm utilized for searching/matching patterns in the text with the help of a hash function. Unlike the Naïve String-Matching algorithm, it doesn't traverse through every character in the initial phase. It filters the characters that do not match and then performs the comparison.

A hash function is a utility to map a larger input value to a smaller output value. This output value is known as the Hash value.

The Rabin-Karp Algorithm is utilized to find all the occurrences of a provided pattern 'P' in a given string 'S' in O(Ns + Np) time, where 'Ns' and 'Np' are the lengths of 'S' and 'P', respectively.

Let us consider the following example to understand it in a better way.

Suppose a string S = "pabctxjabcjfsabcd" and pattern P = "abc". We are required to find all the occurrences of 'P' in 'S'.

We can see that "abc" is occurring in "pabctxjabcjfsabcd" at three positions. Thus, we will return that the pattern 'P' is occurring in string 'S' at indices 1, 7, and 13.

Step 1: Suppose that the given text string is:



Figure 1: The Given Text String

and the pattern to be searched in the above text string is:



Figure 2: The Pattern

Step 2: Let us start by assigning a numerical value (v)/weight for the characters we will use in the problem. Here, we have selected the first ten alphabets only (i.e., A to J).



Figure 3: Text Weights

Step 3: Let x be the length of the pattern, and y be the length of the text string. Here, we have y = 10 and x = 3. Moreover, let n be the number of characters present in the input set. Since we have taken input set as {A, B, C, D, E, F, G, H, I, J}. Therefore, n will be equal to 10. We can assume any preferable value for n.

Step 4: Let us now start calculating the hash value of the pattern:



Figure 4: Hash Value of the Pattern

*Hash Value of Pattern (P) = Σ(v * ny - 1) mod 13*
*= ((3 * 10²) + (4 * 101) + (4 * 10⁰)) mod 13*
= 344 mod 13
= 6

Note: In the above calculation, one can select a prime number in such a manner that we can perform all the calculations with single-precision arithmetic. Here, we have selected 13 as the prime number. We will discuss the purpose of calculating the modulus later.

Step 5: We will now calculate the hash value for the text window of size y.
For the first window ABC,
*Hash Value of text string (s) = Σ(v * nx - 1) mod 13*
*= ((1 * 10²) + (2 * 101) + (3 * 10⁰)) mod 13*
= 123 mod 13
= 6

Step 6: We will now compare the hash value of the pattern with the hash value of the text string. If they match, we will perform the character-matching operation.

In the above examples, the hash value of the first window (i.e., s) matches with the hash value of the pattern (i.e., P). Therefore, we will start matching the characters between ABC and CDD. Since the pattern does not match the first window, we will move to the next window.

Step 7: We will calculate the hash value of the next window by subtracting the first term and adding the next term, as shown below:

$s = ((1 * 10^2) + ((2 * 10^1) + (3 * 10^0)) * 10 + (3 * 10^0)) \bmod 13$
$= 233 \bmod 13$
$= 12$
We will use the former hash value in the following way in order to optimize this process:

$s = ((n * (t - v[\text{character to be eliminated}] * h) + v[\text{character to be included}]) \bmod 13$
$= ((10 * (6 - 1 * 9) + 3) \bmod 13$
$= 12$
Where, $h = n^{y-1} = 10^{3-1} = 100$
Step 8: For BCC, the hash value, s, will become 12, which is not equal to the hash value of the pattern, P, i.e., 6. Therefore, we will move to the next window. After performing some searches, we will manage to find the match for the window CDD in the text string.

Step 1: We will start by defining a function (e.g., rabinKarpSearchAlgo()) for the implementation of the Rabin-Karp Algorithm. This function will accept two arguments - the string 'S' and the pattern 'P'. Firstly, we will calculate the lengths of 'S' and 'P'.

Step 2: We will now select a prime number and a value for the calculation of the modulus while estimating the hash values. To minimize the hashing collision, we will take the prime number value close to the number of characters utilized in the string and pattern. Assuming that the given string 'S' and pattern 'P' contain only lower alphabets, the number of characters will be 26. Therefore, we will take 31 as the prime number. We will now take a value of modulus that should be quite large and prime number. Therefore, we will take the value of modulus as 1e + 9.

Step 3: We will be using the following hash function:

1. hash(S) = (Σ((S[i] - 'a' + 1) * (P^(i)))) % mod

Step 4: We will now create a vector in order to store the powers of "prime" and store (prime ^ 0) to (prime ^ Ns). We will also calculate the hash value of the given pattern and the first window of the string 'S'.

Step 5: At last, we will slide the given pattern one by one and calculate the hash value of the corresponding sub-string or window, comparing it with the hash value of the pattern. If the match is found successfully, we will print the occurrence of the pattern at that index.

Harspool Algorithm

